Car Rental Company

Final Project Report

Team members:

Tymoteusz Kłoska

Szymon Molicki

Mateusz Znaleźniak

Marcjan Zawadzki

Filipa Delfino

Academic year: 2025/2026

Section: Informatics, Section 6

Supervisor: Hafedh Zghidi

Date: July 13, 2026

Contents

Introduction

Project Goal

The goal of the project was to design and implement a database-driven information system supporting the operation of a car rental company.

The system provides a web-based platform that enables customers to browse available vehicles, make reservations, complete online payments, manage their profiles, and access their rental history. At the same time, it offers administrative tools for managing the vehicle fleet, customers, rentals, discount coupons, and invoices.

Car rental companies require centralized information systems to coordinate reservations, maintain accurate information about vehicle availability, process customer payments, generate invoices, and support day-to-day administrative operations. Without such a system, managing concurrent reservations, customer records, and vehicles distributed across multiple locations becomes inefficient and error-prone.

The project emphasizes the practical application of database systems by integrating a relational database with a modern backend API and a web-based frontend. Particular attention was given to maintaining data consistency, enforcing business rules, preventing conflicting reservations for the same vehicle, and providing secure authentication and authorization mechanisms.

Project Scope

The implemented system, named **Metrocars**, simulates the operation of a modern car rental company with multiple rental locations within the GZM metropolitan area. The application models the complete rental process, beginning with customer registration and vehicle selection and ending with payment processing and invoice generation.

Metrocars supports both customer-facing services and the administrative activities required to operate a rental business. Customers can search for available vehicles, create reservations, complete online payments, and access their rental history. Administrators are provided with tools for managing the vehicle fleet, monitoring reservations, maintaining customer information, and managing promotional discount campaigns.

From the database perspective, the project simulates the design and implementation of a relational database supporting a real-world business process.

The implemented system consists of three main components:

- a PostgreSQL relational database responsible for storing all application data,
- a FastAPI backend exposing REST API endpoints and implementing business logic,
- a React frontend providing an intuitive graphical user interface.

Together, these components create an integrated information system that supports the complete rental workflow while ensuring reliable communication between the user interface, application logic, and database layer.

Target Users

The system is designed for two primary groups of users:

- **Customers**, who use the application to:
  - create and manage personal accounts,
  - browse available vehicles,
  - reserve cars,
  - complete online payments, optionally using promotional discount codes,
  - review rental history,
  - download invoices.
- **Administrators**, who are responsible for:
  - managing the vehicle fleet,
  - managing customer accounts,
  - monitoring reservations,
  - managing promotional discount campaigns,
  - maintaining the overall operation of the rental company.

Functional Scope

The implemented functionality reflects the typical workflow of a modern car rental company. The system supports both customer-oriented services and administrative operations, providing tools required to manage the complete rental process from reservation creation to invoice generation.

Customer Features

The customer part of the system enables users to independently complete the entire rental process through a web interface. Customers can create accounts, browse available vehicles, reserve cars, complete payments, and access information about previous rentals without requiring direct assistance from company staff.

The implemented features include:

- **User registration and authentication**

  Customers can create personal accounts and securely access the application. Authentication is based on JSON Web Tokens (JWT), allowing users to manage their personal information and maintain their accounts throughout the entire rental process.

  - creation of a new customer account,
  - secure login using JWT authentication,
  - profile management and editing personal information,

- **Vehicle browsing**

  Before making a reservation, customers can browse the available fleet and compare vehicles based on their characteristics, rental category, pricing, and pickup location. The presented information allows customers to select a vehicle appropriate for their needs.

  - displaying the list of available cars,
  - viewing vehicle details including model, category, and pricing.

- **Rental management**

  The reservation module guides customers through the rental process while ensuring compliance with business rules. Vehicle availability is verified for the selected rental period, preventing multiple customers from reserving the same vehicle simultaneously. Rental costs are calculated automatically based on the selected dates and vehicle pricing.

  - creating new rental reservations,
  - selecting pickup and return locations,
  - specifying rental dates,
  - automatic calculation of rental costs.

- **Online payment**

  After creating a reservation, customers can complete payment directly through the application. The payment process supports promotional discount coupons, validates their eligibility, and records payment information required for invoice generation.

  - payment for reservations,
  - support for multiple payment methods,
  - application of discount coupons during payment,
  - automatic validation of coupon eligibility.

- **Rental history**

  The application stores information about both current and completed rentals, allowing customers to review previous reservations and download invoices generated for paid rentals.

  - viewing previous and current rentals,
  - downloading generated invoices for completed and paid rentals.

Administrator Features

The administrative panel centralizes the operational activities required to manage the rental company. It provides access to business data and management tools necessary to maintain the vehicle fleet, customer accounts, reservations, and promotional campaigns.

The implemented administrator functionalities include:

- **Fleet management**

  Administrators maintain an up-to-date representation of the company's vehicle fleet by adding new vehicles, modifying existing records, and removing vehicles that are no longer available for rental.

  - adding new vehicles,
  - editing vehicle information,
  - removing vehicles from the fleet,
  - viewing complete vehicle details.

- **Customer management**

  Customer management tools provide administrators with access to registered user accounts and their associated information. This functionality supports customer service and ensures that customer records remain accurate and up to date.

  - browsing registered customers,
  - viewing customer information,
  - removing customer accounts when necessary.

- **Rental management**

  Administrators can supervise all reservations created within the system, monitor their current status, and access invoices associated with completed rentals. These tools support the day-to-day operation of the rental business.

  - viewing all rentals within the system,
  - monitoring reservation status,
  - accessing invoices associated with rentals.

- **Discount management**

  Promotional campaigns are managed through configurable discount coupons. Administrators can introduce new promotions, modify existing offers, and remove discounts that are no longer valid.

  - creating promotional discount codes,
  - modifying existing discounts,
  - deleting expired or unused promotions,
  - viewing available discount campaigns.

- **Lookup data**

  The administrative interface provides access to shared lookup data used throughout the system. These predefined values ensure consistency across different parts of the application and simplify administrative data entry.

  - accessing shared lookup data used throughout the administrative interface,
  - supporting consistent selection of predefined values required by the system.

The combination of customer-facing and administrative functionality enables Metrocars to simulate the operation of a real-world car rental company. The implemented features demonstrate how a database-driven information system can coordinate reservations, maintain data consistency, support business processes, and provide separate interfaces tailored to different groups of users.

Technologies and System Architecture

This section describes the technical foundation of the Car Rental Company system, detailing the chosen technology stack, the high-level system architecture, and the communication protocols between components.

Technology Stack

The system is built using modern, industry-standard technologies to ensure scalability, security, and developer productivity. The stack is divided into three primary layers:

- **Backend:**
  - **Python 3.12:** The core programming language.
  - **FastAPI:** A modern, high-performance web framework for building APIs with Python based on standard Python type hints.
  - **SQLAlchemy:** An SQL toolkit and Object-Relational Mapper (ORM) that provides a full suite of well-known enterprise-level persistence patterns.
  - **Alembic:** A lightweight database migration tool for usage with SQLAlchemy.
  - **Argon2id:** Used for secure password hashing via the Passlib library.
- **Frontend:**
  - **React:** A JavaScript library for building user interfaces.
  - **Vite:** A build tool that provides a faster and leaner development experience for modern web projects.
- **Data & Infrastructure:**
  - **PostgreSQL:** A powerful, open-source object-relational database system.
  - **Docker & Docker Compose:** Used for containerization and orchestration of the various services to ensure environment consistency.

System Architecture

The Car Rental Company application follows a **Client-Server architecture** orchestrated via Docker containers. The system is decomposed into three main services that interact within a virtual network.

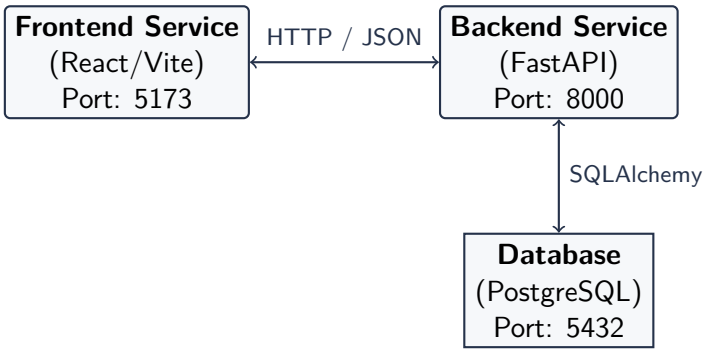

Figure 1: High-level System Architecture Diagram

3.2.1 Containerization

The application is managed using `docker-compose.yml`, which defines three services:

1. **Frontend:** Serves the React single-page application (SPA).
2. **Backend:** Runs the FastAPI server, handling business logic and API requests.
3. **DB:** A PostgreSQL instance that persists all system data.

Authentication and Security

The system implements **JWT (JSON Web Token)** Bearer authentication.

- **Identity Management:** When a user logs in, the backend validates credentials using Argon2id hashing.
- **Token Issuance:** Upon successful validation, the server issues a JWT signed with a `SECRET_KEY`.
- **Authorization:** Subsequent requests include the token in the `Authorization` header. The backend uses FastAPI Dependency Injection to verify roles (Customer vs. Admin).

Database Persistence and Migrations

Data integrity is managed through SQLAlchemy models. To handle evolution of the data schema without data loss, **Alembic** is used for migrations.

```
# Applying migrations to the database
docker compose exec backend alembic upgrade head
```

Listing 1: Example of a Database Migration Command

The system also includes a deterministic seeding script (`seed.py`) that populates the database with 12 locations, 50 cars, and 100 customers, ensuring a consistent testing environment across different developer machines.

API Design

The API is designed following **RESTful principles**. It provides separated endpoints for public, customer, and administrative access.

- **Public:** Login and registration.
- **Customer:** Managing profile, booking rentals, and downloading invoices.
- **Admin:** Full CRUD (Create, Read, Update, Delete) operations on fleet, customers, and discount codes.

Documentation is automatically generated via **Swagger UI**, available at the `/docs` endpoint of the backend service.

Database Design

Database Diagram

The database was designed using the relational model and normalized to reduce data redundancy while maintaining data integrity. The central entity of the schema is the RENTAL table, which connects customers, vehicles, rental locations and the rental lifecycle.

The database consists of transactional entities, such as RENTAL and INVOICE, together with several lookup tables that store predefined values for vehicle characteristics, rental states and payment information. This approach simplifies maintenance, improves consistency, and allows new values to be introduced without modifying the application logic.

Figure 2 presents the complete database schema.

Main Entities

The database is composed of several groups of entities that together implement the complete rental process.

- **Customer management**
  - CUSTOMER stores personal information, authentication data and driving licence details.
  - ADDRESS stores reusable address information shared by customers and rental locations.
- **Rental locations**
  - LOCATION represents company branches where vehicles are stored, collected and returned.
- **Fleet management**
  - CAR stores all information describing a vehicle, including technical parameters, daily rental rate and its current location.
  - Lookup tables (CAR_TYPE, FUEL_TYPE, TRANSMISSION, CAR_STATUS) provide normalized descriptions of vehicle properties.
- **Rental processing**
  - RENTAL represents reservations and active rentals, linking customers, vehicles and pick-up/return locations.
  - RENTAL_STATUS stores predefined rental lifecycle states such as Reserved, Active, Completed or Cancelled.
- **Payments and invoicing**
  - INVOICE stores financial information associated with a rental, including calculated amounts and payment details.
  - PAYMENT_METHOD, PAYMENT_STATUS and INVOICE_STATUS normalize payment-related information.
  - DISCOUNT stores promotional discount codes together with their validity periods and percentage values.

Together, these entities provide complete support for customer management, vehicle management, reservation handling and financial settlement.

Relationships Between Entities

The relationships between entities reflect the business workflow of the rental company.

- One `ADDRESS` may be referenced by multiple customers or rental locations, while every customer and location is associated with exactly one address.
- Each `CAR` belongs to one current rental location and references exactly one fuel type, transmission type, vehicle category and operational status.
- A customer may create multiple rentals, whereas each rental belongs to exactly one customer and one vehicle.
- Every rental specifies one pickup location and one return location. Both relationships reference the `LOCATION` entity.
- Each rental has one rental status describing its current stage within the rental lifecycle.
- Every completed payment generates one invoice associated with exactly one rental.
- An invoice references one payment method, one payment status and one invoice status. Optionally, it may also reference a discount coupon if one was applied during payment.
- Lookup tables are connected using one-to-many relationships, ensuring that frequently reused values remain normalized and consistent throughout the database.

Overall, the schema follows standard relational database design principles. The use of primary keys, foreign keys and lookup tables minimizes redundancy, preserves referential integrity and supports efficient querying of rental, customer and payment data.

Database Implementation

The database was implemented using PostgreSQL, with SQLAlchemy providing the Object-Relational Mapping (ORM) layer between the application and the database. Each table from the relational schema is represented by a corresponding ORM model defining primary keys, foreign keys and relationships.

Database schema changes are managed with Alembic migrations, allowing the schema to evolve in a controlled and reproducible manner without recreating the database. Frequently reused values, such as vehicle types, rental statuses, payment methods and fuel types, are stored in dedicated lookup tables to reduce redundancy and improve consistency.

Referential integrity is enforced through foreign key constraints, while additional business rules, including vehicle availability, rental validation and payment processing, are implemented in the backend before transactions are committed. Together with the deterministic seeding script, this provides a consistent and reproducible database environment for development and testing.

Backend Implementation

The backend was implemented as a FastAPI application responsible for exposing the REST API, validating rental rules, communicating with the PostgreSQL database and keeping reservations, payments and invoices consistent. The code is divided into several main layers: `api` contains HTTP endpoints, `models` contains SQLAlchemy ORM classes, `schemas` contains Pydantic request and response models, `services` contains reusable business logic, and `core` contains infrastructure code such as configuration, security, database sessions, and time helpers.

The application entry point creates the FastAPI instance, configures CORS for the frontend and registers routers for authentication, customer operations, administrator operations and lookup data. Database access is handled through SQLAlchemy sessions injected into endpoints with FastAPI dependencies. Schema evolution and lookup-table seed data are managed with Alembic migrations.

4.5.1 Database Communication

Communication with the database is implemented with SQLAlchemy. The backend reads the database connection string from the `DATABASE_URL` environment variable, creates a shared SQLAlchemy engine and then creates short-lived sessions for individual requests. Endpoints receive the session through the `get_db` dependency, which ensures that the session is closed after the request is processed.

```
DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Listing 2: Database engine and request-scoped session

The same session is passed to service functions when more complex business logic is required. This allows a single operation, such as creating a reservation or processing a payment, to read data, validate relationships, insert records and commit changes as one transaction. For read and write operations the backend uses SQLAlchemy ORM models, while migrations and initial lookup data are managed separately with Alembic.

4.5.2 Request and Response Schemas

The API contract is separated from database models with Pydantic schemas. Request schemas define what data the frontend is allowed to send, while response schemas control which fields are returned to the client. This keeps SQLAlchemy models internal to the backend and provides an additional validation layer before data reaches business logic or database operations.

4.5.3 Authentication and Authorization

Authentication is based on JWT access tokens. Passwords are hashed with Argon2. Protected endpoints use dependencies to check the token payload. Customer routes require a customer account,

while administrator routes require the `admin` role. This keeps access control close to the API layer and avoids duplicating role checks inside every endpoint.

```python
def get_current_payload(credentials=Depends(bearer_scheme)) -> dict[str, Any]:
    try:
        return jwt.decode(credentials.credentials, SECRET_KEY, algorithms=[ALGORITHM])
    except InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid or expired token")


def require_admin(payload=Depends(get_current_payload)) -> dict[str, Any]:
    if payload.get("role") != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return payload


def require_customer(payload=Depends(get_current_payload)) -> dict[str, Any]:
    if payload.get("account_type") != "customer":
        raise HTTPException(status_code=403, detail="Customer access required")
    return payload
```

Listing 3: JWT payload validation and role guards

4.5.4 Customer Registration and Validation

Customer registration creates both the customer account and the related address record. Before saving data, the backend normalizes the e-mail address, checks whether it is already registered, hashes the password, validates the customer's age and verifies the driver's license data. The same validation service is reused later during profile updates and rental creation, which keeps customer-related rules consistent across the application.

4.5.5 Reservation Flow

The most important customer operation is creating a rental reservation. The endpoint validates the requested date range, rejects rentals in the past, checks the customer's age and driver's license, locks the selected car row, verifies that the car is not in an excluded status and checks whether there are conflicting rentals for the same period.

```python
@router.post("/rent", response_model=RentalResponse, status_code=status.HTTP_201_CREATED)
def rent_car(payload: CarRentalRequest, current_customer=Depends(get_current_customer), db=
    Depends(get_db)) -> Rental:
    now = utc_now()
    # ... validate dates, customer age, driver's license and locations ...

    car = _get_locked_car_or_404(db, payload.car_id)
    if db.get(CarStatus, car.car_status_id).name in EXCLUDED_CAR_STATUSES:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Car is not available
    for rental")

    start_datetime, planned_end_datetime = normalize_rental_day_range(...)
    availability = check_car_availability_for_period(db, payload.car_id, start_datetime,
    planned_end_datetime, now=now)
    if not availability.is_available:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Car is not available")

    rental = Rental(
        customer_id=current_customer.customer_id,
        car_id=payload.car_id,
        # ... locations, reserved status and date range ...
```

```
    )
    db.add(rental)
    db.commit()
    return rental
```

Listing 4: Creating a rental reservation

4.5.6 Rental Lifecycle

Rental state is controlled by a service module instead of being hard-coded in individual endpoints. A newly created unpaid rental receives the `reserved` status and blocks the car only for a short payment hold. Paid reservations block the period permanently, active rentals block the car immediately and expired unpaid reservations are canceled automatically.

```
ACTIVE_STATUS = "active"
RESERVATION_HOLD_MINUTES = 5


def check_car_availability_for_period(db: Session, car_id: UUID, start_date: datetime,
    planned_end_date: datetime) -> CarAvailabilityResult:
    now = utc_now()
    conflicts = db.execute(...).all()
    paid_ids = _paid_rental_ids(db, [rental.rental_id for rental, _ in conflicts])

    for rental, status_name in conflicts:
        if status_name == ACTIVE_STATUS or rental.rental_id in paid_ids:
            return CarAvailabilityResult(is_available=False)
        if is_reservation_hold_active(created_at=rental.created_at, now=now):
            return CarAvailabilityResult(is_available=False)
        # ... expired unpaid reservation is marked as cancelled ...

    return CarAvailabilityResult(is_available=True)
```

Listing 5: Core availability rule for rentals

The same service also updates paid rentals according to dates: future rentals remain `reserved`, currently running rentals become `active`, finished rentals become `completed`. This makes the rental history reflect current business state.

4.5.7 Payments and Invoices

Payment finalizes a reservation. The backend verifies ownership of the rental, checks the payment method, rejects expired unpaid reservations, calculates the rental price from the number of days and the car's daily rate, applies an optional discount, creates an invoice and updates the rental lifecycle. Invoice PDF generation is implemented separately with ReportLab and is available as a downloadable `pdf` response.

```
@router.post("/payment", response_model=InvoiceResponse)
def pay_for_rental(payload: RentalPaymentRequest, current_customer=Depends(get_current_customer)
    , db=Depends(get_db)) -> Invoice:
    rental = db.get(Rental, payload.rental_id)
    if rental is None or rental.customer_id != current_customer.customer_id:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Rental not found")

    now = utc_now()
    if not is_reservation_hold_active(created_at=rental.created_at, now=now):
        # ... mark rental as cancelled ...
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Reservation expired")
```

```
    car = db.get(Car, rental.car_id)
    rental_days = count_rental_days(rental.start_date, rental.planned_end_date)
    base_amount = (car.daily_rate * rental_days).quantize(Decimal("0.01"))
    discount_amount = Decimal("0.00")
    # ... validate optional discount code and calculate discount_amount ...

    invoice = Invoice(
        rental_id=rental.rental_id,
        total_amount=base_amount - discount_amount,
        paid_at=now,
        # ... payment method, invoice number and statuses ...
    )

    db.add(invoice)
    db.commit()
    apply_paid_rental_lifecycle_status(db, rental, now=now)
    db.commit()
    return invoice
```

Listing 6: Payment and invoice creation

4.5.8 Consistency

The backend uses multiple consistency mechanisms: Pydantic schemas validate request payloads, SQLAlchemy transactions group related database changes, service functions enforce domain rules and database constraints catch duplicates or invalid relations. Invalid operations are translated into appropriate HTTP responses, for example `400 Bad Request` for validation errors, `404 Not Found` for missing records, `403 Forbidden` for unauthorized access, and `409 Conflict` for unavailable cars or expired reservations.

Frontend Implementation

The frontend of Car Rental Company is a high-performance Single Page Application (SPA) built with **React 18** and **Vite**. The implementation emphasizes a modular component architecture, smooth visual feedback, and a clear separation between administrative and customer interfaces.

4.6.1 Routing and Access Control

The application utilizes `react-router-dom` for navigation. Access to specific parts of the system is managed through higher-order components called **Route Guards**:

- **AppRouteGuard:** Restricts access to authenticated users based on their role (`customer` or `admin`).
- **GuestRoute:** Redirects authenticated users away from login and registration pages to their respective dashboards.

4.6.2 Role-Based Functional Views

The frontend differentiates between consumer-facing and management-facing logic:

- **Customer Experience:** Focused on the `CarsPage`, which features a responsive grid of "Car Cards." Each card is interactive, allowing users to open a detailed modal or jump directly into the multi-step `UserRentPaymentPage` using React Router's `state` to pass car data.
- **Administrative Oversight:** Modules like `AdminUsersPage` provide CRUD (Create, Read, Update, Delete) capabilities. Administrative actions, such as user deletion, are protected by `ConfirmActionModal` to prevent accidental data loss.

The routing structure is divided into three primary modules:

1. **Public:** Home, Login, and Signup pages.
2. **Customer:** Car browsing (`/cars`), personal profile management, rental history, and the multi-step payment flow.
3. **Admin:** Dashboard for fleet management, customer oversight, reservation tracking, and discount code configuration.

4.6.3 Core Layout and Component Architecture

The application utilizes a "Shell" pattern to maintain a consistent user experience across different views.

- **PageShell:** A top-level wrapper that manages the primary navigation (`site-top-panel`) and global decorative elements. It uses role-based logic to conditionally render navigation links for *Customers* (e.g., Profile, History, Cars) or *Admins* (e.g., Users, Reservations, Coupons).
- **PageSection:** A structural component that standardizes page headers, providing consistent "eyebrows," titles, and action areas for all system modules.
- **Modal Orchestration:** The system uses specialized modals like `CarModal` and `EntityDetailsModal`. A notable implementation detail is the "Safe Value Unwrapping" logic, which ensures that complex data objects (such as nested JSON for car brands or fuel types) are correctly rendered as strings without causing React rendering errors.

4.6.4 State Management and Session Persistence

User sessions are managed using React's `useState` and `useEffect` hooks. Authentication tokens are persisted in local storage. The application monitors session expiration and includes a global event listener for `metrocars:unauthorized` events to automatically log users out if their token becomes invalid.

4.6.5 Data Persistence and API Integration

Communication with the FastAPI backend is centralized through a `requestJson` utility. This helper manages:

- **JWT Injection:** Automatically attaches the Bearer token from the session state to every outgoing request.
- **Error Handling:** Provides fallback messaging and captures backend exceptions to update the UI's `error` state variables.

- **Loading States:** Every data-driven view (like `CarsPage` or `AdminUsersPage`) manages a `loading` boolean to show visual feedback during network latency.

4.6.6 UI/UX Design and Visual Identity

The interface employs a distinctive "Metro" theme characterized by a high-contrast palette (`#2C2B2C` and `#FFE75C`). Key visual features include:

- **Dynamic Backdrop:** A `BoardBackdrop` component generates a procedurally placed grid of car SVGs and company logos that scales with the window size.
- **Motion Strips:** CSS-animated "motion strips" on the sides of the car browsing page simulate a road environment, enhancing the thematic consistency of the rental experience.
- **Interactive Components:** Custom-styled "button-pills" and "car-cards" provide a tactile feel with hover transformations and depth-based box shadows.

4.6.7 Responsive Design and Theming

The system uses a custom CSS architecture defined in `index.css`. It leverages CSS Variables (e.g., `-panel-accent`) and `backdrop-filter: blur()` to create a modern, layered glassmorphism effect. The application uses a mobile-first approach with CSS Grid and Flexbox. Using `clamp()` functions and media queries, the layout adapts from a multi-column desktop view (including the decorative motion strips) to a streamlined single-column view for mobile devices, ensuring accessibility across all platforms.

4.6.8 Dynamic Visuals and Parallax Effects

A key aesthetic feature of Car Rental Company is the `CarMotionStrip`. Unlike static images, this component implements a scroll-driven parallax effect:

- It utilizes **requestAnimationFrame** for high-performance 60fps animations.
- The system calculates the scroll position with a `0.35` multiplier to create a sense of depth as cars move vertically in the background.
- **GPU Acceleration:** The use of `translate3d(0, y, 0)` ensures that the animation is offloaded to the browser's hardware acceleration, preventing layout thrashing during heavy scrolling.

```
function updatePosition() {
  const loopHeight = track.scrollHeight / 2;
  // Calculate offset based on scroll position for parallax effect
  const travel = (window.scrollY * 0.35 * direction) % loopHeight;
  const offset = travel < 0 ? travel + loopHeight : travel;
  // Apply hardware-accelerated transform
  track.style.transform = `translate3d(0, ${-offset}px, 0)`;
}
```

Listing 7: High-Performance Scroll Animation Logic

4.6.9 Interactive Features and Easter Eggs

To enhance user engagement, the application includes hidden interactive elements. A specialized **MetroŻubr Easter Egg** is implemented via an invisible trigger on the interface's right margin, which opens a modal featuring unique brand-specific content. This demonstrates the use of React Portals and conditional rendering for non-intrusive UI additions.

```
<Routes>
  <Route path="/" element={<HomePage session={session} />} />
  <Route path="/admin/cars" element={
    <AppRouteGuard session={session} allowedRoles={["admin"]}>
      <AdminCarsPage />
    </AppRouteGuard>
  } />
  <Route path="/user/rent/payment" element={
    <AppRouteGuard session={session} allowedRoles={["customer"]}>
      <UserRentPaymentPage />
    </AppRouteGuard>
  } />
</Routes>
```

Listing 8: Snippet of App Routing and Guards

Test Data

To simplify development and provide a reproducible testing environment, the project includes a deterministic database seeding script (`seed.py`). The script automatically populates the database with representative records covering all major entities and their relationships.

The generated dataset consists of:

- 12 rental locations distributed across cities in the Silesian region,
- 50 vehicles representing different categories, fuel types and transmission types,
- 100 customer accounts with complete personal information,
- 150 rentals covering completed, active and future reservations,
- invoices generated for rentals together with payment information,
- promotional discount codes,
- all lookup tables required by the application.

The lookup tables are initialized first and include predefined values for vehicle status, vehicle type, fuel type, transmission type, rental status, invoice status, payment status and payment methods.

Vehicles are generated using predefined model templates that specify attributes such as brand, model, fuel type, transmission, seating capacity and daily rental rate. Each vehicle is assigned a unique VIN number, registration plate, mileage and rental location.

Customer records contain realistic personal information including addresses, e-mail addresses, phone numbers and driving licence information. Passwords are automatically hashed using the Argon2 algorithm before being stored in the database.

Rental records are generated to represent different stages of the rental lifecycle. Completed rentals include invoices and payment information, active rentals occupy vehicles currently marked as rented,

while future reservations simulate upcoming bookings. Rental prices are calculated automatically based on the vehicle's daily rate and rental duration. Selected rentals additionally receive promotional discounts.

Because the seed script is deterministic, every developer obtains exactly the same dataset after initialization, making application testing, demonstrations and debugging fully reproducible.

Running the Application

Runtime Environment

The application is intended to be started as a Docker Compose environment. This approach keeps the frontend, backend and database configuration consistent across development machines and does not require manual installation of PostgreSQL, Python dependencies or Node.js packages on the host system.

The local environment consists of three services:

- `frontend` – the React/Vite client application exposed on port 5173;
- `backend` – the FastAPI application exposed on port 8000;
- `db` – the PostgreSQL database exposed on port 5432.

Before starting the system, Docker and Docker Compose must be available on the workstation. Docker can be downloaded from the official documentation page: https://docs.docker.com/get-started/get-docker/. After installation, the environment can be verified with the `docker --version` and `docker compose version` commands.

All commands used to run the project should be executed from the root directory of the project repository.

Environment Configuration

The backend reads sensitive and environment-specific settings from a local `.env` file. A template file is provided in the repository and can be copied before the first run:

```
cp .env.example .env
```

The required variables are:

- `SECRET_KEY` – the key used for signing authentication tokens,
- `ADMIN_USERNAME` – the default administrator login,
- `ADMIN_PASSWORD_HASH` – the Argon2id hash of the administrator password.

For local development and presentation purposes, the example configuration provides an administrator account with the username `admin` and the password `admin`. In a production deployment, these values should be replaced with secure, environment-specific credentials.

Starting the Services

The complete application stack can be built and started with one command:

```
docker compose up --build
```

Docker Compose builds the backend and frontend images, starts the PostgreSQL container, waits until the database is healthy, and then starts the backend and frontend services. The backend container is configured to use the internal Docker database address `db:5432`, while the services remain accessible from the host machine through their mapped ports.

Database Initialization

After the containers are running, the database schema must be created by applying Alembic migrations inside the backend container:

```
docker compose exec backend alembic upgrade head
```

The project also includes an optional seed script that prepares the database for local demonstration and testing:

```
docker compose exec backend python seed.py
```

Accessing the Application

Once the services are running and the database has been initialized, the system can be accessed through the following local addresses:

- frontend application: `http://localhost:5173`,
- API root: `http://localhost:8000`,
- Swagger API documentation: `http://localhost:8000/docs`.

The Swagger interface is useful for verifying backend availability and for testing individual API endpoints independently from the frontend.

Stopping and Resetting the Environment

The running containers can be stopped without removing the database volume by using:

```
docker compose down
```

If a clean database state is required, the PostgreSQL volume can be removed as well:

```
docker compose down -v
```

After removing the volume, the application should be started again, migrations should be applied, and the seed script should be executed once more. This recreates the complete local environment from an empty database.

Summary

Completed Work

The objective of the project was to design and implement a database-driven information system supporting the operation of a car rental company. This objective was successfully achieved through the development of **Metrocars**, a complete web application consisting of a PostgreSQL database, a FastAPI backend, and a React frontend.

The implemented system supports the complete rental workflow, including customer registration, authentication, vehicle browsing, reservation management, online payments, invoice generation, and rental history. In addition, an administrative interface was developed to manage vehicles, customers, reservations, and promotional discount campaigns.

From the database perspective, the project demonstrates the design of a normalized relational schema capable of supporting real-world business processes while maintaining data integrity through constraints, relationships, and application-level business rules. The integration of the database with the REST API and graphical user interface illustrates the practical application of database technologies in a multi-tier information system.

The use of Docker Compose, Alembic migrations, and deterministic database seeding further improves the reproducibility of the development environment and simplifies deployment and testing.

Problems Encountered

The primary challenge encountered during the project concerned the design of the database schema. Following the initial review, it became apparent that several attributes had been modeled directly within existing tables even though they were better represented as separate entities connected through foreign key relationships. In addition, some values that were originally stored within operational tables were identified as suitable candidates for dedicated lookup (dictionary) tables in order to improve normalization and maintain consistency.

To address these issues, the database schema was redesigned and a revised version was prepared. The corrected schema was subsequently accepted and served as the foundation for the implementation of the database and application.

Another challenge involved learning how to correctly integrate the database with the backend application in practice. This required gaining practical experience with SQLAlchemy as the Object-Relational Mapper (ORM), configuring the connection between the FastAPI application and the PostgreSQL database, and using Alembic to manage database schema migrations. Understanding how these technologies work together was an important part of the implementation process and provided valuable experience in developing database-driven applications.

Future Improvements

Although the implemented system satisfies the functional requirements defined for the project, several extensions could further improve its capabilities.

Possible future developments include:

- integration with external payment providers instead of the simulated payment process,
- email notifications for reservation confirmations, payment receipts, and rental reminders,

- support for uploading vehicle photographs and additional documentation,
- advanced vehicle search with filtering and sorting based on multiple criteria,
- implementation of customer reviews and vehicle ratings,
- analytical dashboards presenting statistics on rentals, vehicle utilization, and company revenue.

Overall, the project demonstrates the complete development process of a modern database-driven information system. It combines relational database design, backend business logic, REST API development, frontend implementation, authentication mechanisms, and containerized deployment into a cohesive application that reflects the operation of a real-world car rental company.

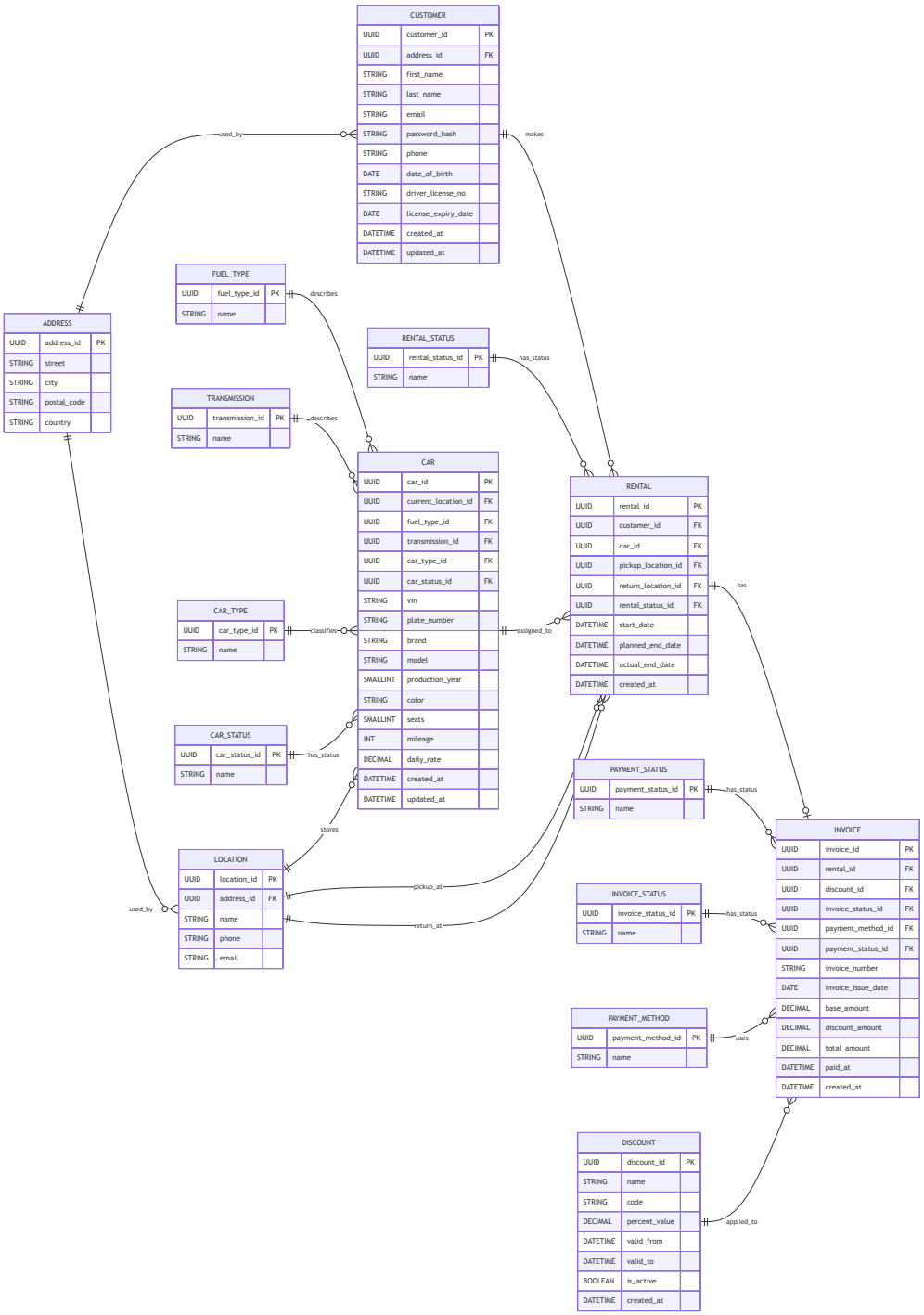

Figure 2: Relational database schema of the Car Rental Company system.